



KTH Technology and Health

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL, CMEDT -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	2023-10-26 08:00-13:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt och hälften av poängen på B-delen samt att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. B-delen rättas endast om minst hälften av poängen på A-delen uppnåtts. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>Kontrollera att projektet hamnar under enheten H på ditt tentamenskonto när du skapar projektet. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg9, uppg10, ...</i>

Mac-användare: För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent, t.ex. "AltGr" + "{" (7:ans tangent).

Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. På fritextfrågor ska svaren motiveras. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

I uppgift 1–3 är *endast ett* alternativ korrekt.

1. Vad innebär, i språket Java, om en datamedlem deklarerats som `protected`?
Datamedlemmen är ...
A. skyddad och går inte att komma åt utanför klassen där den deklarerats
B. går endast att komma åt i klassen den är deklarerad samt i subklasser
C. går endast att komma åt i klassen den är deklarerad, i subklasser samt i klasser i samma paket
D. går endast att komma åt i klassen den är deklarerad samt i klasser i samma paket
(2 p)
2. Vilket av nedanstående påståenden om hantering av undantag är *felaktigt*.
A. Ett undantag, exception, som kastas i en metod kan fångas på vilken nivå som helst anropskedjan
B. Om ett undantag som kastats inte fångas på någon nivå i anropskedjan avbryts exekveringen av applikationen
C. En metod som kan kasta ett undantag måste alltid ha tillägget `throws SomeException` i metodsignaturen, oavsett vilken typ av undantag som kastas
D. En metod kan potentiellt kasta flera olika typer av undantag, men vid en specifik exekvering av metoden kommer högst ett undantag kastas.
(2 p)
3. `String s; s = new String("Java"); s = s.toUpperCase();`
`String t = s; t.toUpperCase();`
Hur många objekt kommer att skapas när satserna ovan exekveras? Du kan behöva läsa dokumentationen för klassen `String`.
A. 1 B. 2 C. 3 D. 4 E. 5
(2 p)

I uppgift 4 ska du ange vilka alternativ, noll till flera, som är korrekta.

4. Antag att en metod, `foo`, implementeras i en klass `Super`, och omdefinieras (`override`) i en subklass, `Sub`. Ett objekt skapas med satsen `Super superRef = new Sub();` Vad är korrekt?
A. `superRef.foo();` innebär att superklassens version av metoden `foo` exekveras
B. Subklassens version av `foo` kan, om så behövs, anropa superklassens version
C. Det går, under exekveringen, inte att via programkod undersöka vilken typ av objekt referensen `superRef` refererar
D. Vid omdefinieringen (`override`) av `foo` i subklassen får antalet argument, och deras typ, ändras förutsatt att metodens namn och returtyp inte ändras
(2 p)

```

5. public class Foo {
    private List<String> strings;
    ...
    public Foo(List<String> initialStrings) {
        this.strings = initialStrings;
        ...
    }

```

Listan strings i Foo ovan ska inte vara möjlig att manipulera annat än via metoder i klassen. Vad är, i det avseendet, problemet med konstruktorn? Hur ska konstruktorn skrivas för att undvika problemet?

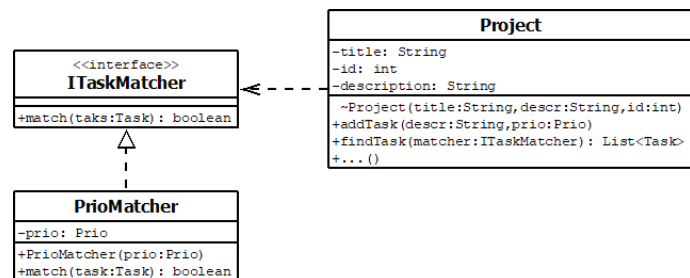
(2 p)

6. Klassdiagrammet beskriver en del i en applikation som hanterar projekt.

a) Förklara vad symbolen mellan PrioMatcher och ITaskMatcher innebär.

b) Förklara vad symbolen mellan Project och ITaskMatcher innebär.

(2 p)



7. När man skriver objektorienterad kod är det önskvärt dels att klasser (och paket av klasser) har hög kohesion och dels att man har låg koppling mellan klasser. Förklara varför designmönstret Model-View-Controller kan anses uppfylla

a) hög kohesion,

b) låg koppling.

8. I koden nedan används två trådar för att summera ett stort antal värden parallellt. Klassen ThreadSum har en run-metod som summerar och en metod, getSum, som ger resultatet.

```

SumThread thread1 = new SumThread(firstHalfOfValues);
SumThread thread2 = new SumThread(secondHalfOfValues);
thread1.start(); thread2.start();
int sum = thread1.getSum() + thread2.getSum();

```

Koden ovan kommer inte att ge den korrekta summan. Varför?

Uppdatera kodstycket med det som saknas så att resultatet, summan, blir korrekt. API-dokumentationen om klassen Thread kan vara till hjälp.

(2 p)

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" et cetera. Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på `P:/Java/api`.

Alla klasser i denna del av tentamen ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha meningsfulla konstruktorer, som korrekt initierar alla datamedlemmar, access-metoder samt en omdefinierad version av `toString`.

9. Uppdrag

Skriv en klass, `Task`, som representerar information om ett uppdrag, något som ska utföras. Ett objekt av typen `Task` har en text, en prioritet samt ett datum för senaste uppdatering. Prioriteten representeras med enum med värdena `låg`, `normal` och `hög`. Senaste uppdatering representeras med `java.time.LocalDate`.

Det ska vara möjligt att uppdatera prioritet samt att lägga till ytterligare text till den redan existerande. Det ska inte finnas någon publik `set`-metod för senaste uppdatering, men datamedlemmen ska uppdateras internt varje gång något data i objektet ändras.

Klassen ska implementera det generiska interfacet `java.util.Comparable`. Rangordningen ska i första hand bestämmas av prioriteten, men om dessa är lika, i andra hand av texten (alfabetisk ordning). Metoden `equals` ska omdefinieras så att två objekt anses lika om de har samma prioritet och text.

Skriv en `main`-metod där du skapar objekt och demonstrerar funktionaliteten. Du kan demonstrera uppgift 9 och 10 i samma `main`.

(4 p)

10. Uppdragslista

För att hantera många uppdrag behöver vi en klass, `TaskList`. Klassen ska ha en intern lista med uppdrag, `tasks`. Klassen ska ha två konstruktorer, en som skapar en tom lista och en som tar en lista med existerande uppdrag som argument. Den senare konstruktorn ska kontrollera om argumentet är `null` eller ej. Om argumentet är `null` ska ett `IllegalArgumentException` kastas, i annata fall ska uppdragen föras över till den interna listan.

Nedanstående metoder ska finnas.

- `createTask(text, priority)` och `createTask(text)`, som skapar ett nytt task och lägger det i listan, den senare versionen sätter `normal` prioritet
- `findTask(...)`, som tar en textsträng som argument och returnerar en lista med matchande objekt. Om argumentet ingår som del i uppdragets text anses det matcha.
- `removeTask(Task task)`, som tar bort task ur listan och returnerar `true`, alternativt returnerar `false` om task inte finns i listan.
- `getAllTasks`, som returnerar en kopia av listan med uppdrag.

I kontraktet för klassen ingår att objekten i listan alltid ska vara sorterade enligt den rangordning som definierats i klassen `Task`. Klassen `java.util.Collections` har metoder för sortering.

Skriv även en main-metod som demonstrerar funktionaliteten.

(4 p)

11. StringBuilder

I denna uppgift ska du skriva en förenklad version av klassen `StringBuilder`, för ändringsbara textsträngar. Klassen ska internt ha en array av typ `char[]`. När den interna arrayen är full ska den automatiskt utökas, vilket implementeras i en privat hjälpmetod.

Klassen ska ha nedanstående funktionalitet.

- En parameterslös konstruktör samt en konstruktör som tar en textsträng (`String`) som argument. I det senare fallet ska tecknen i argumentet överföras till den interna arrayen
- `length`, som returnerar aktuellt antal tecken
- `capacity`, som returnerar den interna arrayens storlek
- `charAt(index)`, som returnerar det indikerade tecknet eller kastar ett `IndexOutOfBoundsException`
- `append(char c)`, som lägger till `c` i slutet av existerande text
- `append(String str)`, som lägger till tecknen från `str` i slutet av existerande text
- `toString`, som returnerar ett `String`-objekt med motsvarande text – använd `new String(theArray, 0, n)` där `n` står för antal tecken

Skriv sedan en main-metod som demonstrerar funktionaliteten.

(4 p)

12. Funktionellt och lambda

Skriv ett generiskt interface, `Func`, med en metod, `R apply(A object)`. Typparametrarna `A` och `R` representerar argumentets typ respektive returtypen. Implementationer av interfacet `Func` ska kunna användas för att mappa ett objekt mot ett nytt, utifrån data i det första objektet.

Skriv en implementation, `HasHighPriorityMapper`, där metoden `apply` tar ett `Task`-objekt som argument och returnerar ett `Boolean`-objekt med värdet `true` om prioriteten är hög, annars `false`.

Skriv sedan en separat klass med en generisk klassmetod, `map`, som tar som argument en lista med objekt av någon typ och en implementation av interfacet `Func`. Metoden ska returnera en ny lista med nya objekt utifrån implementationen av `Func`. Signaturen för metoden ska vara `public static <A, R> List<R> map(List<A> objects, Func<A, R> function)`

Skriv sedan en main-metod där du skapar en Lista med `Task`-objekt och demonstrerar

1. metoden `map` tillsammans med `HasHighPriorityMapper`,
2. metoden `map` tillsammans med ett lambda-uttryck som motsvarar en metod som mappar ett `Task`-objekt mot objektets prioritet,
3. metoden `map` tillsammans lista av `String`-objekt och ett lambda-uttryck som motsvarar en metod som mappar ett `String`-objekt mot textens längd.

(4 p)